Project

ระบบวางแผนเมนูอาหารและบริหารจัดซื้อผลผลิตชุมชนด้วยปัญญาประดิษฐ์

Our Team

นายกรวิชญ์ นิจรัญ 002

นางสาวชยานันท์ ชนสูงเนิน 043

นางสาวตรีทิพย์ สุไลมาน 088

นายธีรธัฐฐิ์ วิบูลสันติ 115

นางสาวธัญชนก ธรรมบัวชา 108

2.ความเป็นมาของโครงงาน

ปัจจุบัน การวางแผนเมนูอาหารและการจัดซื้อวัตถุดิบยังพบปัญหาการเลือกเมนูไม่ตรงกับวัตถุดิบที่มีอยู่ การจัดซื้อเกินความจำเป็น ทำให้สิ้นเปลืองงบประมาณและเกิดวัตถุดิบเหลือทิ้ง อีกทั้งเกษตรกรในชุมชนยังขาดช่องทางจำหน่ายผลผลิตที่เหมาะสม

คณะผู้จัดทำจึงพัฒนาระบบวางแผนเมนูอาหารและบริหารจัดซื้อผลผลิตชุมชนด้วยปัญญาประดิษฐ์ เพื่อช่วยวิเคราะห์วัตถุดิบ จำนวนผู้บริโภค และผลผลิตที่มีจำหน่าย พร้อมแนะนำเมนู คำนวณปริมาณวัตถุดิบ และวางแผนการจัดซื้อให้เหมาะสม

โครงงานนี้ช่วยลดต้นทุนและเวลาในการวางแผนอาหาร ลดปัญหาวัตถุดิบเหลือทิ้ง เพิ่มประสิทธิภาพในการจัดซื้อ และส่งเสริมการนำผลผลิตจากชุมชนมาใช้ให้เกิดประโยชน์ พร้อมทั้งเพิ่มช่องทางจำหน่ายและสร้างรายได้ให้แก่เกษตรกรในชุมชน

3.วัตถุประสงค์ของโครงงาน

1.1 เพื่อพัฒนาระบบสำหรับจัดเก็บและค้นหาข้อมูลวัตถุดิบ สูตรอาหาร เกษตรกร และข้อมูล การจัดซื้อให้เป็นระบบเดียวกัน

1.2 เพื่อคำนวณปริมาณวัตถุดิบที่ต้องใช้และปริมาณวัตถุดิบที่ต้องจัดซื้อเพิ่มเติมตามจำนวน อาหารที่ต้องการผลิต

1.3 เพื่อส่งเสริมการสร้างรายได้ให้แก่เกษตรกรในชุมชนผ่านการวางแผนจัดซื้อผลผลิตอย่างเป็น ระบบ

4.ขีดความสามารถ และแสดงตัวอย่างของโครงงานที่ทำเสร็จแล้ว

1. แสดงวัตถุดิบคงเหลือน้อย วัตถุดิบที่ใกล้วันที่ควรใช้ก่อน และคำสั่งซื้อที่รอรับ
2. เพิ่ม แก้ไข ค้นหา รับเข้า เบิกใช้ และบันทึกของเสีย โดยแยกเป็นล็อต
3. บันทึกชื่อเมนู รายการวัตถุดิบและปริมาณต่อจาน เพิ่มและแก้ไขสูตรได้
4. เก็บเกษตรกร ผลผลิต ราคา ปริมาณพร้อมขาย และวันที่พร้อมส่ง
5. กำหนดเงื่อนไข รับคำแนะนำ เลือกเมนู ระบุจำนวนจาน และดูรายการที่ต้องซื้อ
6. สร้างคำสั่งซื้อ เลือกผู้ขาย บันทึกรับสินค้า และเพิ่มเข้าคลัง
7. ดูยอดจัดซื้อ การเบิกใช้ ของเสีย และมูลค่าซื้อจากเกษตรกรตามช่วงเวลา

5.โมเดลที่ใช้ในการพัฒนาซอฟต์แวร์ พร้อมหน้าที่ของแต่ละคนตามโมเดล

รอบพัฒนา

รอบ 1 ฐานข้อมูล คลังวัตถุดิบ และสูตรอาหาร

รอบ 2 ข้อมูลเกษตรกร ผลผลิต และการจัดซื้อ

รอบ 3 แผนเมนู การคำนวณ และการเชื่อม AI

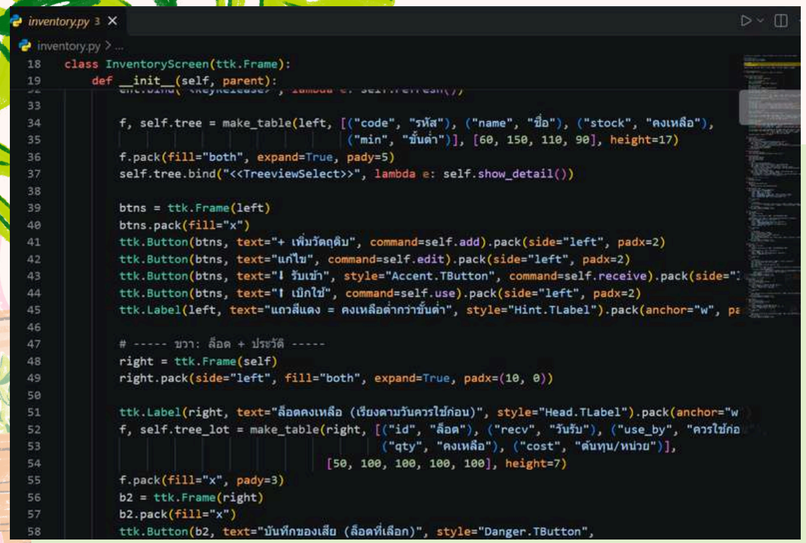
รอบ 4 รายงาน ทดสอบรวม และเตรียมสาธิต

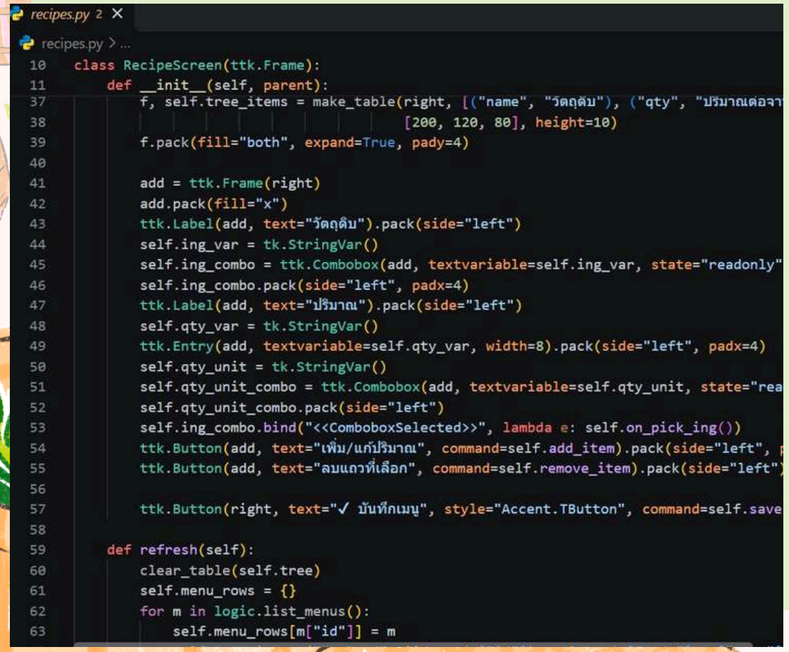
หน้าที่	
นายธีรธัฐฐ์ วิบูลสันติ	พัฒนาระบบ
นายกรวิชญ์ นิจรัญ	ทดสอบระบบ
นางสาวธัญชนก ธรรมบัวชา	เอกสาร
นางสาวชยานันท์ ชนสูงเนิน	รูปเล่ม
นางสาวตรีทิพย์ สุไลมาน	เอกสารนำเสนอ

6. โครงสร้างโครงงานและการเก็บข้อมูล

กลุ่มข้อมูล	ข้อมูล
วัตถุดิบ	รหัส ชื่อ หน่วยมาตรฐาน และระดับคงเหลือขั้นต่ำ
ล็อตวัตถุดิบ	วัตถุดิบ วันที่รับเข้า วันที่ควรใช้ก่อน จำนวนคงเหลือ และต้นทุนต่อหน่วย
ประวัติเคลื่อนไหว	รับเข้า เบิกใช้ ของเสีย ปรับยอด วันเวลา และรายการอ้างอิง
เมนูและสูตร	ชื่อเมนู วัตถุดิบ และปริมาณต่อจาน
เกษตรกร	ชื่อกลุ่มหรือผู้ขาย พื้นที่ และข้อมูลติดต่อ
ผลผลิตพร้อมขาย	ผู้ขาย วัตถุดิบ ราคา ปริมาณ และวันที่พร้อมส่ง
แผนเมนู	วันที่ เมนู จำนวนจาน งบ และสถานะ
คำสั่งซื้อและรายการซื้อ	ผู้ขาย วัตถุดิบ จำนวน ราคา และสถานะรับสินค้า

7.โค้ดโดยสรุป

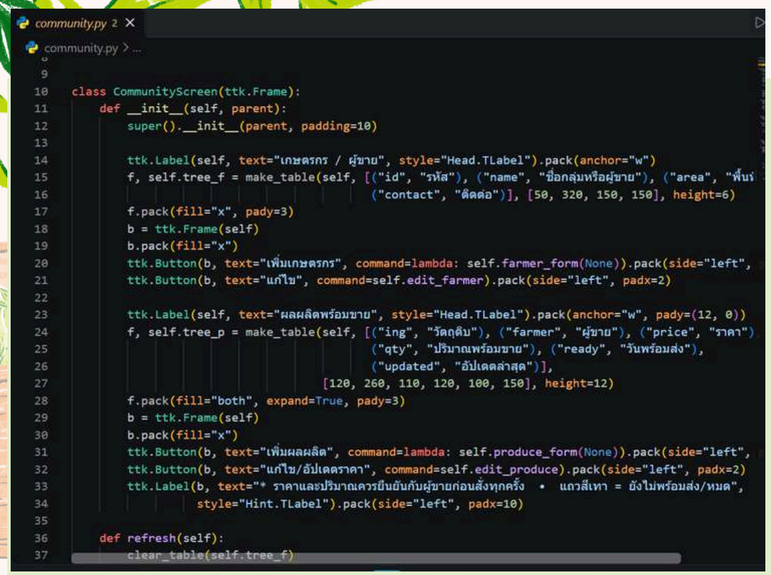




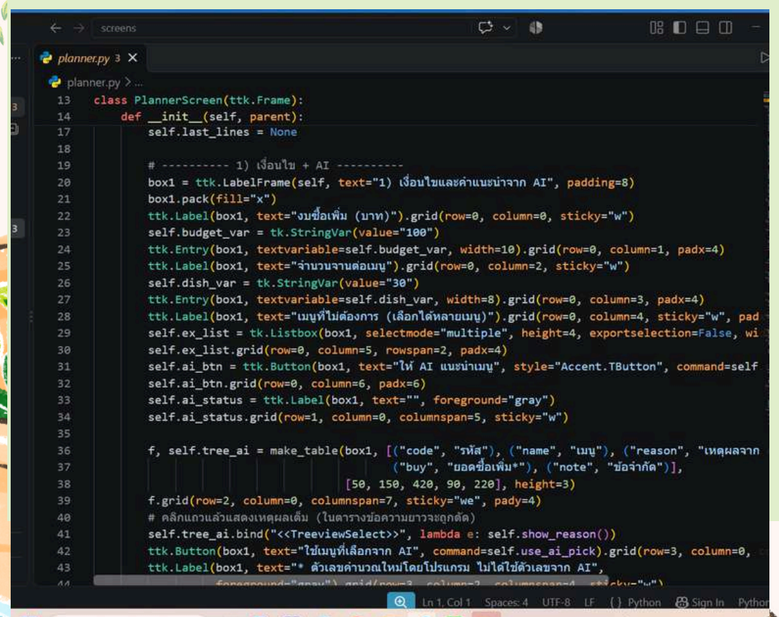
inventory.py เป็นส่วนหน้าจอจัดการคลังวัตถุดิบ ใช้แสดงข้อมูลและจัดการวัตถุดิบผ่านช่องกรอกข้อมูล ตาราง และปุ่มต่าง ๆ เช่น เพิ่มข้อมูล รับเข้า และเบิกใช้ โดยระบบจะตรวจสอบและบันทึกข้อมูลลงฐานข้อมูล แล้วแสดงข้อมูลล่าสุดในตาราง เพื่อให้ผู้ใช้จัดการคลังวัตถุดิบได้สะดวกและถูกต้อง

recipes.py คือส่วนหน้าจอจัดการสูตรอาหาร ใช้กำหนดว่าแต่ละเมนูมีวัตถุดิบอะไร และใช้ปริมาณเท่าไรต่อหนึ่งจาน เช่น ฟักทองผัดไข่หนึ่งจานใช้ฟักทอง 100 กรัม และไข่ 1 ฟอง ข้อมูลสูตรนี้จะนำไปใช้คำนวณวัตถุดิบเมื่อวางแผนทำอาหารหลายจาน

7.โค้ดโดยสรุป



```python
class CommunityScreen(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)

        ttk.Label(self, text="เกษตรกร / ผู้ขาย", style="Head.TLabel").pack(anchor="w")
        f, self.tree_f = make_table(self, [("id", "รหัส"), ("name", "ชื่อกลุ่มหรือผู้ขาย"), ("area", "พื้นที่
                                           ("contact", "ติดต่อ")], [50, 320, 150, 150], height=6)
        f.pack(fill="x", pady=3)
        b = ttk.Frame(self)
        b.pack(fill="x")
        ttk.Button(b, text="เพิ่มเกษตรกร", command=lambda: self.farmer_form(None)).pack(side="left",
        ttk.Button(b, text="แก้ไข", command=self.edit_farmer).pack(side="left", padx=2)

        ttk.Label(self, text="ผลผลิตพร้อมขาย", style="Head.TLabel").pack(anchor="w", pady=(12, 0))
        f, self.tree_p = make_table(self, [("ing", "วัตถุดิบ"), ("farmer", "ผู้ขาย"), ("price", "ราคา")
                                           ("qty", "ปริมาณพร้อมขาย"), ("ready", "วันพร้อมส่ง"),
                                           ("updated", "อัปเดตล่าสุด")],
                                    [120, 260, 110, 120, 100, 150], height=12)
        f.pack(fill="both", expand=True, pady=3)
        b = ttk.Frame(self)
        b.pack(fill="x")
        ttk.Button(b, text="เพิ่มผลผลิต", command=lambda: self.produce_form(None)).pack(side="left",
        ttk.Button(b, text="แก้ไข/อัปเดตราคา", command=self.edit_produce).pack(side="left", padx=2)
        ttk.Label(b, text="* ราคาและปริมาณควรยืนยันกับผู้ขายก่อนสั่งทุกครั้ง  •  แถวสีเทา = ยังไม่พร้อมส่ง/หมด",
                  style="Hint.TLabel").pack(side="left", padx=10)

    def refresh(self):
        clear_table(self.tree_f)
```

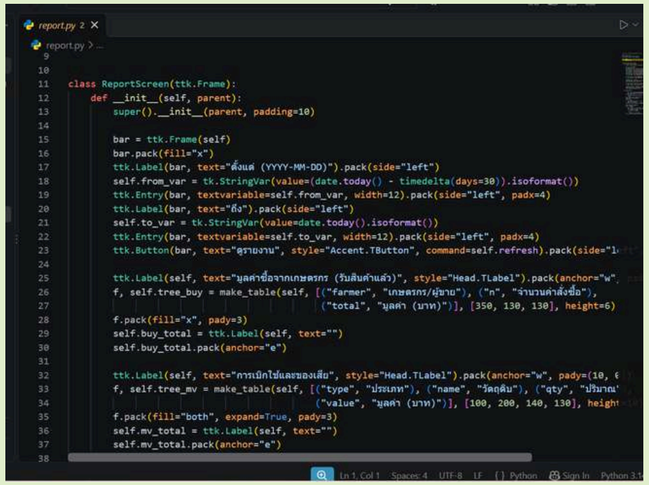


```python
class PlannerScreen(ttk.Frame):
    def __init__(self, parent):
        self.last_lines = None

        # ---------- 1) เงื่อนไข + AI ----------
        box1 = ttk.LabelFrame(self, text="1) เงื่อนไขและคำแนะนำจาก AI", padding=8)
        box1.pack(fill="x")
        ttk.Label(box1, text="งบซื้อเพิ่ม (บาท)").grid(row=0, column=0, sticky="w")
        self.budget_var = tk.StringVar(value="100")
        ttk.Entry(box1, textvariable=self.budget_var, width=10).grid(row=0, column=1, padx=4)
        ttk.Label(box1, text="จำนวนจานต่อเมนู").grid(row=0, column=2, sticky="w")
        self.dish_var = tk.StringVar(value="30")
        ttk.Entry(box1, textvariable=self.dish_var, width=8).grid(row=0, column=3, padx=4)
        ttk.Label(box1, text="เมนูที่ไม่ต้องการ (เลือกได้หลายเมนู)").grid(row=0, column=4, sticky="w", pad
        self.ex_list = tk.Listbox(box1, selectmode="multiple", height=4, exportselection=False, wi
        self.ex_list.grid(row=0, column=5, rowspan=2, padx=4)
        self.ai_btn = ttk.Button(box1, text="ให้ AI แนะนำเมนู", style="Accent.TButton", command=self
        self.ai_btn.grid(row=0, column=6, padx=6)
        self.ai_status = ttk.Label(box1, text="", foreground="gray")
        self.ai_status.grid(row=1, column=0, columnspan=5, sticky="w")

        f, self.tree_ai = make_table(box1, [("code", "รหัส"), ("name", "เมนู"), ("reason", "เหตุผลจาก
                                            ("buy", "ยอดซื้อเพิ่ม*"), ("note", "ข้อจำกัด")],
                                     [50, 150, 420, 90, 220], height=3)
        f.grid(row=2, column=0, columnspan=7, sticky="we", pady=4)
        # คลิกแถวแล้วแสดงเหตุผลเต็ม (ในตารางข้อความยาวจะถูกตัด)
        self.tree_ai.bind("<<TreeviewSelect>>", lambda e: self.show_reason())
        ttk.Button(box1, text="ใช้เมนูที่เลือกจาก AI", command=self.use_ai_pick).grid(row=3, column=0,
        ttk.Label(box1, text="* ตัวเลขคำนวณใหม่โดยโปรแกรม ไม่ได้ใช้ตัวเลขจาก AI",
```

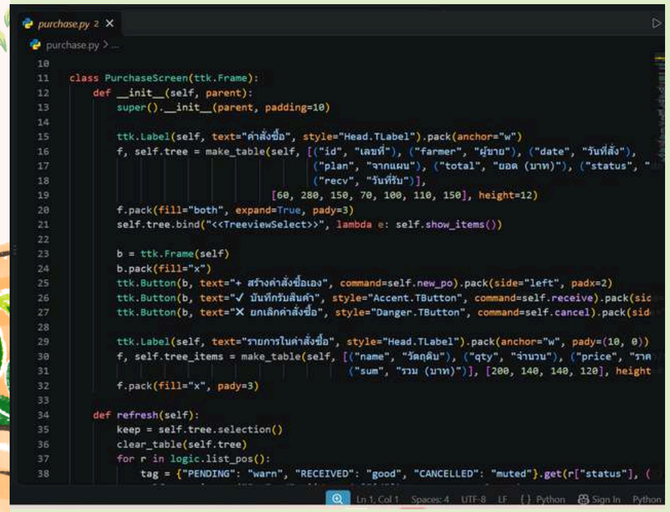
planner.py คือส่วนหน้าจอวางแผนเมนูอาหาร ให้ผู้ใช้กำหนดจำนวนจานและงบประมาณ ขอคำแนะนำเมนู แล้วดูปริมาณวัตถุดิบที่ต้องใช้และรายการที่ต้องซื้อเพิ่ม

community.py เป็นส่วนหน้าจอจัดการข้อมูลเกษตรกรและผลผลิตพร้อมขาย เช่น ชื่อผู้ขาย ราคาวัตถุดิบ ปริมาณที่มี และวันที่พร้อมส่ง เพื่อใช้วางแผนเมนูและจัดซื้อวัตถุดิบจากชุมชน

7.โค้ดโดยสรุป



```python
class ReportScreen(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)

        bar = ttk.Frame(self)
        bar.pack(fill="x")
        ttk.Label(bar, text="ตั้งแต่ (YYYY-MM-DD)").pack(side="left")
        self.from_var = tk.StringVar(value=(date.today() - timedelta(days=30)).isoformat())
        ttk.Entry(bar, textvariable=self.from_var, width=12).pack(side="left", padx=4)
        ttk.Label(bar, text="ถึง").pack(side="left")
        self.to_var = tk.StringVar(value=date.today().isoformat())
        ttk.Entry(bar, textvariable=self.to_var, width=12).pack(side="left", padx=4)
        ttk.Button(bar, text="ดูรายงาน", style="Accent.TButton", command=self.refresh).pack(side="l

        ttk.Label(self, text="มูลค่าซื้อจากเกษตรกร (รับสินค้าแล้ว)", style="Head.TLabel").pack(anchor="w",
        f, self.tree_buy = make_table(self, [("farmer", "เกษตรกร/ผู้ขาย"), ("n", "จำนวนคำสั่งซื้อ"),
                                             ("total", "มูลค่า (บาท)")], [350, 130, 130], height=6)
        f.pack(fill="x", pady=3)
        self.buy_total = ttk.Label(self, text="")
        self.buy_total.pack(anchor="e")

        ttk.Label(self, text="การเบิกไปแลของเสีย", style="Head.TLabel").pack(anchor="w", pady=(10, 6
        f, self.tree_mv = make_table(self, [("type", "ประเภท"), ("name", "วัตถุดิบ"), ("qty", "ปริมาณ'
                                            ("value", "มูลค่า (บาท)")], [100, 200, 140, 130], height
        f.pack(fill="both", expand=True, pady=3)
        self.mv_total = ttk.Label(self, text="")
        self.mv_total.pack(anchor="e")
```



```python
class PurchaseScreen(ttk.Frame):
    def __init__(self, parent):
        super().__init__(parent, padding=10)

        ttk.Label(self, text="คำสั่งซื้อ", style="Head.TLabel").pack(anchor="w")
        f, self.tree = make_table(self, [("id", "เลขที่"), ("farmer", "ผู้ขาย"), ("date", "วันที่สั่ง"),
                                         ("plan", "จากแผน"), ("total", "ยอด (บาท)"), ("status", "
                                         ("recv", "วันที่รับ")],
                                  [60, 280, 150, 70, 100, 110, 150], height=12)
        f.pack(fill="both", expand=True, pady=3)
        self.tree.bind("<<TreeviewSelect>>", lambda e: self.show_items())

        b = ttk.Frame(self)
        b.pack(fill="x")
        ttk.Button(b, text="+ สร้างคำสั่งซื้อเอง", command=self.new_po).pack(side="left", padx=2)
        ttk.Button(b, text="✓ บันทึกรับสินค้า", style="Accent.TButton", command=self.receive).pack(sic
        ttk.Button(b, text="✕ ยกเลิกคำสั่งซื้อ", style="Danger.TButton", command=self.cancel).pack(sid

        ttk.Label(self, text="รายการในคำสั่งซื้อ", style="Head.TLabel").pack(anchor="w", pady=(10, 0))
        f, self.tree_items = make_table(self, [("name", "วัตถุดิบ"), ("qty", "จำนวน"), ("price", "ราค
                                               ("sum", "รวม (บาท)")], [200, 140, 140, 120], height
        f.pack(fill="x", pady=3)

    def refresh(self):
        keep = self.tree.selection()
        clear_table(self.tree)
        for r in logic.list_pos():
            tag = {"PENDING": "warn", "RECEIVED": "good", "CANCELLED": "muted"}.get(r["status"], (
```

purchase.py คือส่วนหน้าจอจัดซื้อวัตถุดิบ ใช้สร้างคำสั่งซื้อ ระบุผู้ขาย รายการวัตถุดิบ จำนวนและราคา รวมถึงบันทึกการรับสินค้าเข้าคลัง

report.py คือส่วนหน้าจอแสดงรายงาน ใช้เลือกช่วงเวลาที่ต้องการ แล้วแสดงสรุปข้อมูล เช่น ยอดจัดซื้อ การเบิกวัตถุดิบ ของเสีย และมูลค่าที่ซื้อจากเกษตรกรในชุมชน

8. ปัญหาและแนวทางแก้ไข

ปัญหาที่อาจพบ

1. AI แนะนำเมนูหรือคำนวณวัตถุดิบผิดพลาด

2. วัตถุดิบไม่เพียงพอหรือเหลือทิ้ง

3. ข้อมูลเกษตรกรไม่เป็นปัจจุบัน หรือ AI ขัดข้อง

แนวทางแก้ไข

1. ตรวจสอบข้อมูลและใช้โปรแกรมคำนวณให้ถูกต้อง

2.ตรวจสอบสต็อกและคำนวณปริมาณก่อนจัดซื้อ

3.ตรวจสอบข้อมูลกับผู้ขาย และเลือกเมนูเองเมื่อ AI ใช้งานไม่ได้

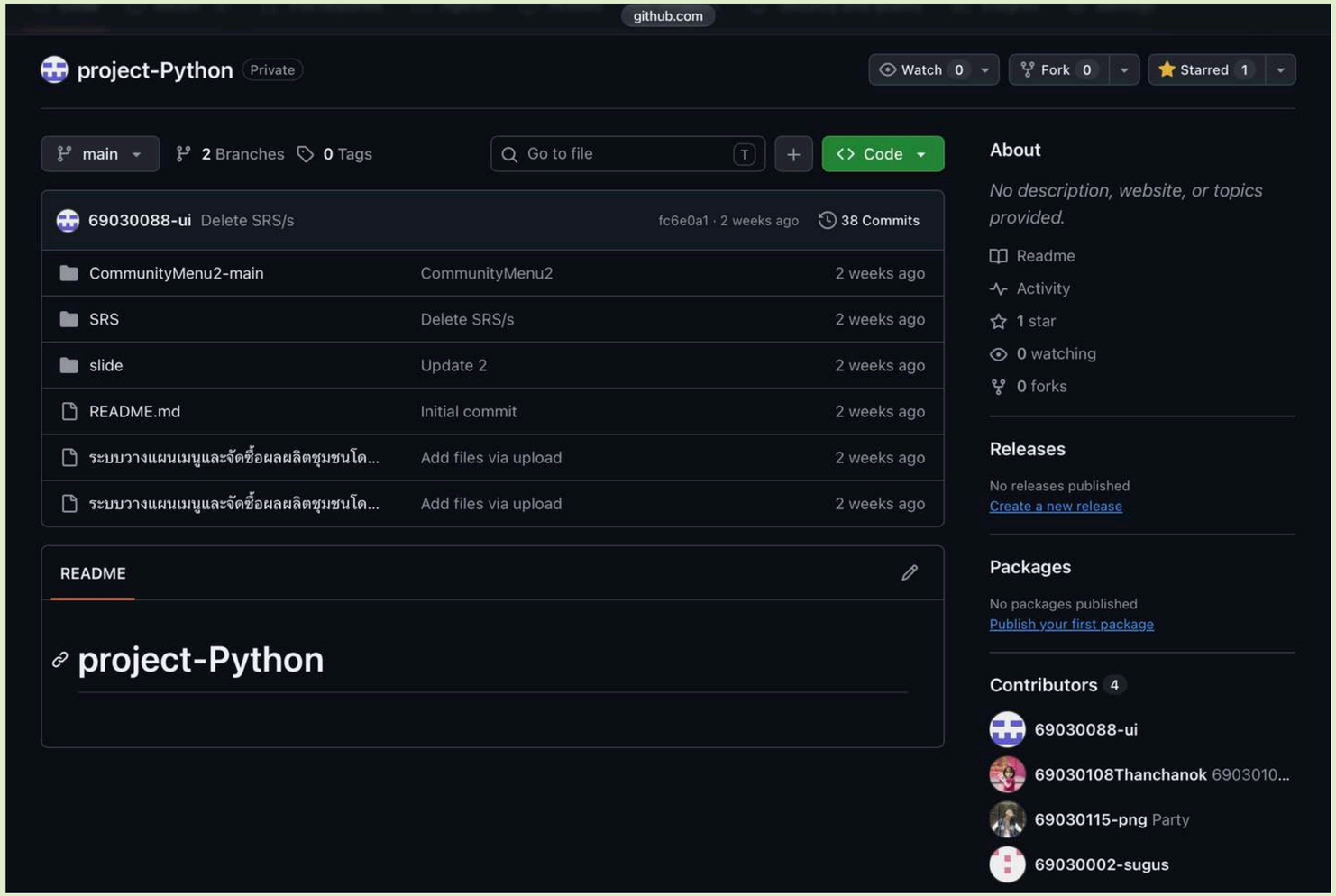
9. การนำไปประยุกต์ใช้งาน

1.ร้านอาหารและโรงอาหาร ใช้ระบบวางแผนเมนูและคำนวณปริมาณวัตถุดิบให้เหมาะสมกับจำนวนผู้บริโภค

2.การจัดซื้อวัตถุดิบ ช่วยตรวจสอบวัตถุดิบคงเหลือและวางแผนจัดซื้อ เพื่อลดต้นทุนและลดวัตถุดิบเหลือทิ้ง

3.การส่งเสริมเกษตรกรในชุมชน เชื่อมโยงผู้ประกอบการกับเกษตรกร เพื่อเพิ่มช่องทางจำหน่ายผลผลิตและสร้างรายได้ให้ชุมชน

10. การทำงานบน GitHub



ระบบวางแผนเมนูอาหารและบริหารจัดซื้อผลผลิตชุมชนด้วยปัญญาประดิษฐ์
Community Menu Planner

Start

Thank You